

TD 10 – University Application Project

Evaluation. These two weeks, you will be working on a small project that will be evaluated. You are required to submit your project as a single `zip` or `tar.gz` file on Moodle (`moodle.di.ens.fr`) before 23:59 on Tuesday, May 5th. The project will be evaluated by running it and making tests on a browser, before looking at the code, so please make sure it runs before submitting. Also, note that Exercises 7 to 11 require a (albeit short) written answer, so please make sure to include it somehow.

1 Goal

The goal of the project is to create a prototype website for managing a university. You will first develop the accounting part of the website, that manages the accounts of students at the local student-run bar. Then, you will take care of the school section of the website that handles students choices of curriculums, courses, and grades. In more details, this site should manage:

- *Students*, each with: a *last name*, a *first name*, and a *phone number*;
- An *account* per student, with a *balance*, that is positive and necessarily less than a billion.
- An history of *deposits* to accounts, keeping the recipient account, the amount, and the time of each deposit.
- An history of *transfers* between accounts, keeping in memory the issuer account, the recipient account and the time of the operation. Transfers are necessarily positive and between different accounts.
- *Teachers*, each with: a *last name*, a *first name*, and a *phone number*;
- *Curriculums*, each with: a *name*, a *director*. Programs are attended by *students* and contain *courses*. Students and courses can be shared among several programs;
- *Courses*, each with: a *name*, a *teacher*, *validations* (e.g., a project, TD, exams). A validation cannot be shared between multiple courses. Each course awards a number of *ECTS credits* depending on the curriculum followed;
- *Validations*, which have a *name* (e.g., midterm, final exam, presentation), a *coefficient*, a *date*, and for each person who takes the exam, a *grade*;

The average grade of a student in a course is the average of the points obtained in that course, weighted by the coefficients (and if a grade is not in the database, the grade obtained by the student is considered as 0). The average grade of a student in a curriculum is the average of the grades obtained in each course, weighted by the ECTS credits (and again, if a student has no grade for a course, their grade is 0).

Exercise 1. Carefully design a schema to model this situation. Amounts of money and grades should be stored using the `DECIMAL` type. Schedule of operations should be stored using `TIMESTAMP`, with default value `NOW()`. Then, create a database on the PostgreSQL server and execute this schema on it. This is the database you will work on for the rest of this project.

2 Web App

You will find a `zip` file containing the skeleton of a Python web app using Flask on Moodle. You will likely have to install additional Python modules (in particular, `flask` and `psycopg2`). You should use `pip` to retrieve and install these packages, through the following commands : `pip install flask` and `pip install psycopg2-binary`. You may also have to install `wtforms` the same way.

This is not necessary to understand for the rest of the project, but if you are interested, the code is organized as follows:

- The file `model.py` is the one that will interest us for this project, and which contains all the functions handling the internal representation of data. This file mostly relies on `psycopg2` to do so.
- Folders `static` and `templates` contain the CSS and HTML code used by flask to generate the frontend of the website. In particular, `layout.html` specifies the content common to all pages (mostly the header), and `listings.html` specifies how to display the result of queries.
- The file `forms.py` is only used to define the forms that will let users add new students, grades and so on, that appear at the bottom of most pages.
- The file `run.py` takes care of the link between the backend and the frontend of the website. Each block starting with `@app.route(...)` specifies what a given page of the website should display, by calling the functions defined in `model.py` and the forms defined in `forms.py`.

You will see in the next course that this way of designing a website is called MVC, for Model-View-Controller.

As said earlier, you will only have to edit the file `model.py`, where `TODO` appears. Your task is then to write SQL queries as specified right before the function, between `self.cursor.execute(f"""` and `""")`. Note that this syntax uses f-strings, introduced in Python 3.6, that let you print the content of a variable in the string by putting it between curly braces, as follows: `f"The variable contains {variable}"`.

You can then run the app by filling in your PostgreSQL credentials on line 10 of `model.py` (in the argument of `psycopg2.connect`), and by running `python3 run.py` in a terminal. You can then access the website at address `http://127.0.0.1:5000/` from your favorite browser. While working, do not hesitate to test often what you wrote, using placeholder (like `SELECT 0, ..., 0 FROM some_table`) for queries that you have not written yet.

Exercise 2. Get the website running.

3 Accounting

As mentioned earlier, we start with the Students and Accounts tabs.

Exercise 3. Fill in the queries `TODO 01` to `TODO 04`. When a full name is asked for, you should return the first name and the last name separated by a space, like “Lucas Larroque” for instance. You can use the operator `||` to append two strings in SQL.

Exercise 4. Fill in the `deposit` and `transfer` functions, as a sequence of SQL queries executed with `self.cur.execute(...)`.

Exercise 5. We now want to enforce another constraint on the transfers: the total amount of money transferred from a given account cannot be greater than 1000 € over a rolling 24 hours period. Fill the function `getTransferredAmountOver24Hours` so that it returns the sum of the transferred amounts from the account given as input during the last 24 hours, plus the additional amount passed as argument. In PostgreSQL, `NOW() - INTERVAL '24' HOUR` returns the current time minus 24 hours.

Then, edit your `transfer` function with a check that the daily limit has not been met, and raise an exception if it has been, with `raise Exception(<appropriate message>)`.

Exercise 6. Add an artificial 5 seconds delay to the function `transfer`, just before and just after the first update command, using `time.sleep(5)`. This artificial delay will allow us to simulate latency in communication networks, the waiting time for confirmation from a user, etc.

3.1 Inconsistencies

Exercise 7. Try to transfer an amount of money from an account to itself, or to transfer more money than available on an account. Any issue? Fix them by editing the `transfer` function to ensure that these cases do not happen: raise an exception with `raise Exception(<appropriate message>)` when one of these situations happen.

Solution 7. The check condition on the *transfer* table is violated (**new row for relation transfer violates check constraint transfer_check**), when transferring money from an account to itself. Nevertheless, the two updates on the *account* table are actually carried out, but they do compensate each other, so the database stays in a consistent state.

When transferring more money than an account has, a check on the *account* table is violated (**new row for relation transfer violates check constraint transfer_check**). Because this is the first instruction, nothing is really carried out through this request.

Exercise 8. Make a transfer in a browser tab, and, in another, display the list of open accounts, refreshing the page regularly. Explain the observed inconsistencies.

Solution 8. The table is in an inconsistent state between both operations, i.e., the total amount of money across all accounts is not preserved during 5 seconds.

Exercise 9. Make a transfer in a browser window, and use `CTRL + C` to interrupt the execution of the Python server at an appropriate time, then restart the Python server. Describe the inconsistent state we obtain.

Solution 9. A failure of the database management system occurring in between both operations leads to a loss of money. More concerning, the transaction is not logged in the table *transfer*.

Exercise 10. Make two transfers at the same time, taking care to reveal an inconsistency. Explain this unwanted behavior.

Solution 10. If we perform, at the same time, two transfers of 1k euro each from the same origin account, both will succeed, breaking the daily limit. That is to say, the two concurrent transactions both read the table *transfer* before they write over it. This situation is known as *phantom reads*¹.

3.2 Transactions

To solve these inconsistencies, we can use the transaction management system of PostgreSQL:

- the starting point of a transaction is given by the order `BEGIN TRANSACTION` ;
- any update order made within a transaction requires a lock on the corresponding rows ;
- locks are released at the end of a transaction; `COMMIT` ends the transaction, `ROLLBACK` cancels the transaction. A failure or a runtime error has the same effect as a `ROLLBACK`.

Exercise 11. Use the transactional system of PostgreSQL to solve two of the three inconsistencies noted previously. What problems are solved?

Solution 11. Problems from Exercises 8 and 9 are solved, but Exercise 10 is not fixed. Refer to the source code under the directory *solution*.

¹<https://www.postgresql.org/docs/current/transaction-iso.html>

Exercise 12. For the most intricate cases, we have to manually lock some tables, with the order `LOCK TABLE table_name`. Use this functionality to fix the remaining issues.

Solution 12. We add a `LOCK TABLE transfer` just after the starting of the transaction. The lock is released automatically when the transaction ends. This does indeed resolve the problem from Exercise 10.

Exercise 13. Fill `TODO 08` to finish the Accounts part. You can use text in apostrophes as a constant to return in SQL, and `UNION` to return the union of several queries.

4 Grading

You now have to fill `TODO 09` to `TODO 39`. Most of them are easy, but a few are a bit more difficult or require more involved structures. It may make sense to keep them for after having taken care of the easy ones. Here are a few SQL functions and constructions that you may find useful for these more difficult queries:

- The construction `WITH name AS (subquery) query` lets you define named subqueries.
- `CROSS JOIN` is Cartesian product. `NATURAL JOIN` is a join over all the attributes sharing the same names between the two tables. `LEFT JOIN` is a join where tuples of the left table are kept in the output even if there is no tuple in the right table satisfying the condition. The corresponding rows are then `NULLs`.
- `query1 EXCEPT query2` returns all the answers of `query1` that are not answers of `query2`.
- `EXISTS query` returns `true` if `query` has at least one answer.
- As mentioned earlier, apostrophes are used as string delimiters, and constants can be returned (by `SELECT 'string' FROM table` for instance). In addition, the operator `||` handles concatenation of strings.
- The function `COALESCE(a, b)` will return `a` if `a` is not `NULL` and `b` otherwise.